

Project Aetherville

LLM 자율 시민, YOLO 비전 자율주행, RL 신호등 최적화가
한 도시에서 동시에 살아 숨쉬는 풀스택 AI 사회 시뮬레이터

- ▶ Generative Agents + Autonomous Driving + Traffic RL
- ▶ 분산 클라우드 추론 · 브라우저 3D 렌더링 · 음성 God Mode
- ▶ 인간 라벨링 0% · 자기 진화 시뮬레이션

과목

파이썬 기반 머신러닝·딥러닝 및 로봇 제어 공학

유형

개인 프로젝트

대상 카테고리

비전 · 딥러닝 · 로봇시뮬 · NLP 통합

인프라

RTX 4090 Cloud · Local Browser Client

목차

01	Executive Summary	03
02	프로젝트 개요와 목표	04
03	과목 카테고리 매핑	05
04	시스템 아키텍처	06
05	기술 스택	08
06	모노레포 디렉토리 구조	09
07	WebSocket 메시지 스키마	11
08	REST API 스펙	14
09	데이터 모델	15
10	모듈 상세 설계	17
11	데이터 · 라벨링 전략	22
12	개발 마일스톤	23
13	인프라 · 배포	27
14	리스크 매트릭스	29
15	평가 지표	30
16	발표 시연 시나리오	31
17	향후 확장	32
18	참고 문헌	33

01. Executive Summary

Project Aetherville는 LLM 기반 자율 시민, 컴퓨터 비전 기반 자율주행 차량, 강화학습 신호등, 시계열 트래픽 예측 모델을 단일 가상 도시 환경에서 통합 운영하는 풀스택 AI 사회 시뮬레이터다.

본 프로젝트의 핵심 차별점은 다음 세 가지다.

1. 인간 라벨링 0%

모든 캐릭터 페르소나, 도시 시설명, 일일 이벤트, 객체인식 학습 데이터는 LLM과 시뮬레이션이 자체 생성한다. 개발자는 아키텍처와 오케스트레이션 로직만 작성한다.

2. 분산 클라우드 추론 아키텍처

모든 AI 추론은 클라우드 GPU 서버(RTX 4090)에서 처리되고, 로컬 PC는 Three.js 기반 3D 렌더링과 UI만 담당한다. WebSocket을 통한 실시간 상태 동기화로 어떤 노트북에서든 시연 가능하다.

3. 창발 행동 (Emergent Behavior)

사전 스크립트 없이 시민들의 메모리·반성·계획 순환을 통해 사회적 사건이 자율 발생한다. 발표자조차 시연 결과를 완전히 예측할 수 없다.

한 줄 정의

"LLM 시민 + 비전 자율주행 + RL 신호등이 단일 가상 도시 안에서 통합 운영되는, 인간 라벨링 없이 자기 진화하는 풀스택 AI 사회 시뮬레이터."

주요 학술 레퍼런스

- Park et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST.
- Ultralytics (2023). YOLOv8: Real-time Object Detection.
- Schulman et al. (2017). Proximal Policy Optimization Algorithms.
- Salimans et al. (2024). vLLM: Easy, Fast, and Cheap LLM Serving.

02. 프로젝트 개요와 목표

2.1 배경

본 과목의 자유 주제 과제는 (1) 객체인식 및 비전, (2) 머신러닝 및 딥러닝, (3) 로봇 제어 및 시뮬레이션, (4) 자연어 처리 및 챗봇의 네 가지 카테고리 중 하나를 선택하도록 권장한다. 본 프로젝트는 단일 카테고리에 안주하지 않고, **4개 카테고리**를 단일 무대 위에서 자연스럽게 통합한다.

2.2 핵심 목표

번호	목표	설명
G1	4 카테고리 유기 통합	가상 도시라는 단일 도메인 위에서 비전·딥러닝·로봇시뮬·NLP가 억지 없이 맞물려 동작
G2	창발 행동 관찰	사전 스크립트 없이 시민 간 사회적 상호작용이 자율 발생
G3	분산 아키텍처	클라우드 추론 + 로컬 렌더링으로 어떤 노트북에서도 시연 가능
G4	제로 라벨링	학습용 데이터 전량을 LLM과 시뮬레이션으로 자동 생성
G5	인터랙티브 시연	음성 명령으로 환경에 개입 가능한 God Mode 인터페이스

2.3 산출물 (Deliverables)

- 모노레포 코드베이스 (Server + Client + Infra)
- Docker Compose 기반 클라우드 배포 스크립트
- 3분 데모 영상 + 라이브 시연용 시나리오 스크립트
- 본 PRD 문서 + 발표용 슬라이드
- GitHub 공개 리포지토리 (README, 아키텍처 다이어그램, 데모 GIF)

2.4 범위 제외 (Out of Scope)

- 실제 차량/로봇 하드웨어 제어 (전량 시뮬레이션)
- LLM 사전학습 또는 파인튜닝 (사전훈련 모델 활용)
- 다관절 매니퓰레이터(Robot Arm) 제어 — 모바일 로봇(차량/드론)만 사용
- VR/AR 인터페이스 (브라우저 기반 데스크탑 시연만)

03. 과목 카테고리 매핑

각 학습 카테고리가 본 프로젝트의 어떤 모듈에 매핑되며, 어떤 산출 자료로 평가될 수 있는지를 정리한다.

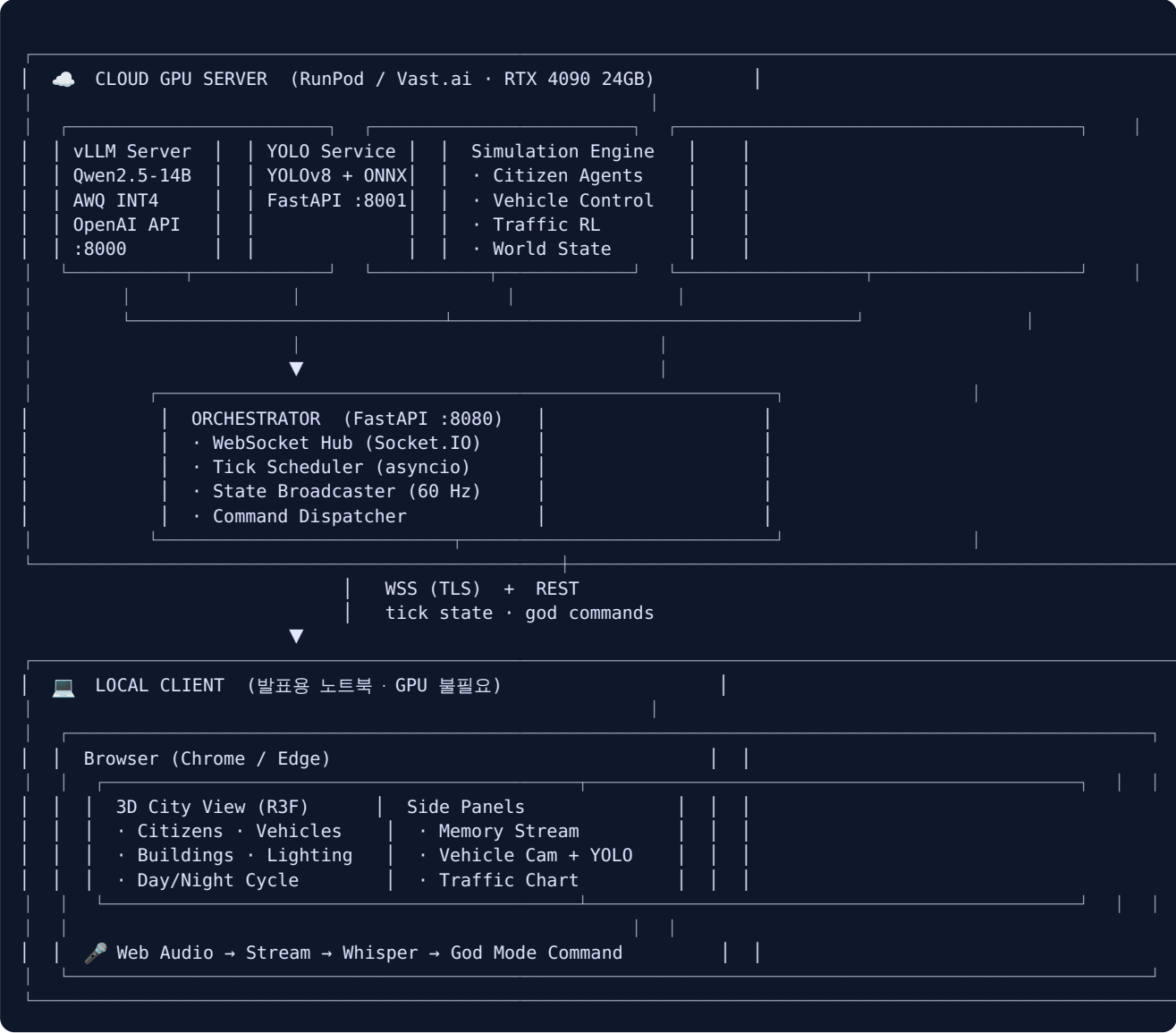
카테고리	담당 모듈	핵심 기술	증빙 산출물
NLP 자연어 처리 / 챗봇	Citizen Agent	vLLM, Qwen2.5-14B, RAG, 메모리 스트림, 반성 알고리즘	시민 대화 로그, 메모리 트레이스, 시민 간 emergent 대화 사례
VISION 객체인식 / 비전	Vision Module	YOLOv8, ONNX Runtime, 자동 라벨링 파이프라인	차량 카메라 실시간 추론 영상, mAP 지표, 자동 생성 데이터셋
ML/DL 머신러닝 / 딥러닝	Traffic AI (LSTM + RL)	PyTorch LSTM, Stable-Baselines3 PPO, 시계열 분석	트래픽 예측 그래프, 신호등 PPO 학습 곡선, ELO 비교
ROBOT 로봇 제어 / 시뮬레이션	Vehicle Control + Drone	Kinematic vehicle model, A* path planning, PID 제어, Bezier 경로	차량 자율주행 영상, 드론 비행 경로, 충돌 회피 로그
EXTRA 시스템 / 인프라	Orchestrator + Infra	FastAPI, WebSocket, Docker, vLLM serving	아키텍처 다이어그램, 배포 스크립트, latency 측정

역지 통합이 없는 이유

"가상 도시"라는 단일 무대가 4개 기술을 자연스럽게 요구한다. 시민이 살려면 NLP가 필요하고, 시민이 출근하려면 차량(로봇 제어)이 필요하며, 차량이 충돌을 피하려면 비전이 필요하고, 도시 트래픽을 관리하려면 ML이 필요하다.

04. 시스템 아키텍처

4.1 고수준 다이어그램



4.2 데이터 흐름 (Tick 단위)

단계	위치	동작
T0	Sim Engine (Cloud)	가상 시계 1 tick 진행 (1 tick = 시물상 1분, 실시간 100ms)
T1	Citizen Agents	현재 시간 기준 plan tree에서 행동 추출, 이동/대화/작업 결정
T2	Vehicle Control	요청된 트립 처리, kinematic 위치 업데이트, A* 경로 따라 이동
T3	Vision Service	차량 카메라 가상 프레임 → YOLO 추론 → 보행자/신호등 박스 반환

단계	위치	동작
T4	Traffic AI	RL 정책으로 신호등 상태 갱신, LSTM 예측치 업데이트
T5	Orchestrator	월드 상태 직렬화 → WebSocket으로 모든 클라이언트에 broadcast
T6	Browser Client	상태 수신 → Three.js scene 업데이트 → 16ms 안에 렌더

4.3 LLM 호출 최적화

20명의 시민이 매 tick마다 LLM을 호출하면 비용과 지연이 폭발한다. 다음 전략으로 호출 빈도를 1/100 수준으로 낮춘다.

- **Plan Caching:** 시민은 하루에 한 번 plan tree를 생성하고, 이후 tick에서는 캐시된 계획만 참조한다.
- **Event-Driven Invocation:** 다른 시민/차량과 충돌·만남이 발생할 때만 LLM 호출 (dialog, replanning).
- **Reflection Batching:** 6시간마다 한 번씩 모든 시민의 reflection을 vLLM 배치 추론으로 처리.
- **Continuous Batching (vLLM):** 여러 시민의 요청을 동시에 묶어 처리, throughput 4~8배 향상.

05. 기술 스택

5.1 백엔드 (Cloud Server)

레이어	기술	버전	비고
LLM 서빙	vLLM	≥ 0.6	OpenAI 호환 엔드포인트, AWQ 양자화
LLM 모델	Qwen2.5-14B-Instruct-AWQ	—	한국어 강함, 14B AWQ로 4090 24GB에 안착
대안 모델	Qwen2.5-7B / EXAONE 3.5	—	backup 또는 빠른 응답용
임베딩	bge-m3	—	한국어 멀티링구얼 임베딩
벡터 DB	ChromaDB	≥ 0.5	로컬 영속화, 시민별 collection
오케스트레이션	FastAPI + asyncio	≥ 0.115	비동기 tick scheduler
WebSocket	python-socketio	≥ 5.11	양방향 실시간 통신
객체인식	YOLOv8 (ultralytics)	≥ 8.3	nano 모델로 충분
추론 가속	ONNX Runtime GPU	≥ 1.19	YOLO 모델 ONNX 변환 후 사용
RL	Stable-Baselines3	≥ 2.3	PPO 알고리즘
시계열	Darts	≥ 0.30	LSTM/TFT 모델 지원
STT	faster-whisper	≥ 1.0	small 모델, GPU 추론
캐시	Redis	≥ 7.4	월드 스냅샷, plan 캐시
컨테이너	Docker + Compose	—	4개 서비스 분리 배포

5.2 프론트엔드 (Local Browser)

레이어	기술	버전	비고
프레임워크	Next.js (App Router)	≥ 15	React 19 기반
3D 렌더링	Three.js	≥ 0.170	WebGL2
React 통합	@react-three/fiber	≥ 9	R3F
R3F Helpers	@react-three/drei	≥ 10	OrbitControls, Sky, Stats, Loader
포스트프로세싱	@react-three/postprocessing	≥ 3	Bloom, SSAO, DepthOfField
상태 관리	Zustand	≥ 5	월드 상태 store

레이어	기술	버전	비고
WebSocket	socket.io-client	≥ 4.8	서버와 동일 프로토콜
차트	Recharts	≥ 2.13	트래픽 그래프
UI 컴포넌트	shadcn/ui + Tailwind	—	사이드 패널, 다이얼로그
음성 입력	Web Audio API + MediaRecorder	—	마이크 스트림
TTS	Web Speech API	—	시민 대화 음성 출력 (선택)
3D 에셋	Kenney City Kit (CC0)	—	로우폴리 건물/차량/캐릭터
캐릭터 애니메이션	Mixamo (무료)	—	걷기/뛰기/대화 애니메이션

5.3 개발 도구

- 패키지 관리: `uv` (Python), `pnpm` (Node)
- 린팅/포맷: `ruff + mypy` (Python), `biome` (TS/JS)
- 스키마 동기화: Pydantic (Python) ↔ Zod (TS) via `datamodel-code-generator`
- CI/CD: GitHub Actions (lint, test, Docker build)
- 모니터링: Prometheus + Grafana (선택)

06. 모노레포 디렉토리 구조

전체 시스템은 **pnpm workspace + uv workspace** 기반 모노레포로 구성한다.

```

aetherville/
├── README.md
├── pnpm-workspace.yaml
├── pyproject.toml
├── docker-compose.yml
├── docker-compose.cloud.yml
├── .env.example
├── .github/
│   └── workflows/
│       ├── lint.yml
│       └── docker-build.yml
├──
├── packages/
│   ├── shared-schemas/
│   │   └── package.json
│   │   └── pyproject.toml
│   │   └── src/
│   │       ├── python/aetherville_schemas/
│   │       │   ├── __init__.py
│   │       │   ├── messages.py
│   │       │   ├── state.py
│   │       │   └── commands.py
│   │       └── typescript/
│   │           ├── messages.ts
│   │           └── state.ts
│   └── scripts/
│       └── generate-ts.sh
├──
├── server/
│   ├── pyproject.toml
│   ├── Dockerfile
│   ├──
│   ├── orchestrator/
│   │   ├── main.py
│   │   ├── ws_hub.py
│   │   ├── scheduler.py
│   │   ├── broadcaster.py
│   │   ├── command_handler.py
│   │   └── config.py
│   ├──
│   ├── sim/
│   │   ├── world.py
│   │   ├── city.py
│   │   ├── pathfinding.py
│   │   └── physics.py
│   ├──
│   ├── agents/
│   │   ├── citizen.py
│   │   ├── memory.py
│   │   ├── reflection.py
│   │   ├── planner.py
│   │   ├── dialog.py
│   │   ├── persona_generator.py
│   │   └── prompts/
│   │       ├── persona.jinja
│   │       ├── plan_day.jinja
│   │       ├── reflect.jinja
│   │       ├── dialog.jinja
│   │       └── action_decide.jinja
│   ├──
│   └── vehicles/
└──

```

uv workspace root

전체 서비스 오케스트레이션

클라우드 배포 오버라이드

서버/클라이언트 공유 스키마

WebSocket 메시지 Pydantic 모델

월드 상태 모델

God Mode 명령 모델

자동 생성된 TS 타입

Pydantic → Zod 자동 변환

클라우드 백엔드

메인 오케스트레이터 서비스

FastAPI 엔트리

Socket.IO 허브

asyncio tick scheduler

60Hz 상태 송출

God Mode 명령 디스패치

시뮬레이션 엔진

World class · 시간/날씨/이벤트

도시 맵 절차적 생성

A* + nav graph

AABB collision, kinematic update

시민 에이전트

Citizen 클래스

Memory Stream + ChromaDB

반성 알고리즘

Plan tree 생성

시민 간 대화

LLM으로 페르소나 자동 생성

프롬프트 템플릿 (jinja2)

차량 + 드론 제어

```

├── vehicle.py          # 차량 클래스 (kinematic)
├── drone.py            # 드론 (Bezier 경로)
├── controller.py       # PID + 경로 추종
├── trip_manager.py     # 시민 픽업/드롭 매칭
├── render_camera.py    # 가상 카메라 → 프레임 생성
├──
├── vision/            # YOLO 추론 서비스
│   ├── service.py      # FastAPI :8001
│   ├── yolo_runner.py  # ONNX Runtime wrapper
│   ├── auto_label.py   # PyBullet-free 자동 라벨링
│   └── models/
│       └── yolov8n_city.onnx # 학습 완료된 모델
├──
├── traffic_ai/        # 트래픽 RL + 시계열
│   ├── env.py          # gym.Env 래퍼 (TrafficSignalEnv)
│   ├── train_ppo.py    # PPO 사전 학습 스크립트
│   ├── inference.py    # 추론 전용 wrapper
│   ├── forecaster.py   # LSTM 트래픽 예측
│   └── checkpoints/
│       └── ppo_traffic.zip
├──
├── llm/               # LLM 클라이언트 어댑터
│   ├── client.py       # vLLM OpenAI 호환 클라이언트
│   ├── embeddings.py   # bge-m3 wrapper
│   └── prompt_loader.py
├──
├── voice/             # 음성 처리
│   ├── stt.py          # faster-whisper
│   └── intent_parser.py # 음성 → God 명령 파싱
├──
├── tests/
│   ├── test_memory.py
│   ├── test_planner.py
│   └── test_pathfinding.py
├── client/            # 로컬 브라우저 프론트엔드
│   ├── package.json
│   ├── next.config.ts
│   ├── tailwind.config.ts
│   ├── public/
│   │   ├── models/    # GLB 3D 에셋
│   │   │   ├── buildings/
│   │   │   ├── vehicles/
│   │   │   └── characters/
│   │   └── textures/
│   ├── src/
│   │   ├── app/       # Next.js App Router
│   │   │   ├── layout.tsx
│   │   │   ├── page.tsx # 메인 시물 화면
│   │   │   └── replay/page.tsx # 오프라인 리플레이 모드
│   │   ├── scene/     # Three.js 씬
│   │   │   ├── CityScene.tsx
│   │   │   ├── Citizen.tsx
│   │   │   ├── Vehicle.tsx
│   │   │   ├── Drone.tsx
│   │   │   ├── Building.tsx
│   │   │   ├── TrafficLight.tsx
│   │   │   ├── SkyDayNight.tsx
│   │   │   └── PostFX.tsx
│   │   └── ui/         # 사이드 패널 UI
│   │       ├── MemoryPanel.tsx
│   │       └── VehicleCamPanel.tsx # YOLO 박스 오버레이

```

```

├── TrafficChartPanel.tsx
├── EventTimeline.tsx
├── GodModeMic.tsx
├── store/                                # Zustand
├── │   ├── worldStore.ts
├── │   ├── selectionStore.ts
├── │   └── connectionStore.ts
├── ws/                                  # 통신 레이어
├── │   ├── client.ts
├── │   ├── handlers.ts
├── │   └── reconnect.ts
├── lib/
├── │   ├── audio.ts                    # Web Audio 스트림
├── │   └── coords.ts                  # 시물 좌표 ↔ 씬 좌표 변환
├── tests/
├── infra/                              # 인프라 코드
├── │   ├── runpod/
├── │   │   ├── deploy.sh              # RunPod 인스턴스 부팅
├── │   │   ├── pod-template.json
├── │   │   └── README.md
├── │   ├── docker/
├── │   │   ├── vllm.Dockerfile
├── │   │   ├── orchestrator.Dockerfile
├── │   │   ├── vision.Dockerfile
├── │   │   └── client.Dockerfile
├── │   └── nginx/
├── │       └── tls.conf                # WSS TLS 종단 (Caddy 대안)
├── scripts/                            # 데이터 생성 · 학습 스크립트
├── │   ├── generate_personas.py        # 시민 페르소나 LLM 생성
├── │   ├── generate_city_map.py        # 도시 질차적 생성
├── │   ├── auto_label_yolo.py          # YOLO 학습 데이터 자동 생성
├── │   ├── train_yolo.py
├── │   ├── train_traffic_ppo.py
├── │   └── benchmark_latency.py
├── docs/                               # 문서
├── │   ├── PRD.pdf                   # 본 문서
├── │   ├── architecture.svg
├── │   ├── demo-scenarios.md
├── │   ├── api-reference.md
├── │   └── slides/
├── │       └── presentation.pdf

```

07. WebSocket 메시지 스키마

모든 WebSocket 메시지는 envelope 패턴을 따른다. 서버 → 클라이언트는 60Hz로 송출되는 상태 업데이트와 비정기 이벤트 알림으로 구성된다.

7.1 메시지 Envelope

```
{
  "v": 1,                      // 프로토콜 버전
  "type": "state_update" | "event" | "command" | "ack" | "error",
  "ts": 1700000000.123,        // unix timestamp (sec, float)
  "tick": 12345,               // 시뮬레이션 tick number
  "payload": { ... }           // 타입별 페이로드
}
```

7.2 서버 → 클라이언트 메시지

state_update (60Hz, 핵심 메시지)

```

{
  "v": 1,
  "type": "state_update",
  "ts": 1700000000.123,
  "tick": 12345,
  "payload": {
    "world": {
      "time_of_day": "07:43",
      "weather": "clear",          // clear | rain | fog | night
      "temperature": 18.5
    },
    "citizens": [
      {
        "id": "c01",
        "name": "김민지",
        "pos": [120.5, 0.0, 80.2], // x, y(height), z
        "rot": 1.57,               // radians
        "anim": "walking",        // idle | walking | running | sitting | talking
        "current_action": "출근 중",
        "talking_to": null        // 다른 시민 id 또는 null
      }
    ],
    "vehicles": [
      {
        "id": "v01",
        "type": "taxi",
        "pos": [220.0, 0.0, 150.0],
        "rot": 0.0,
        "speed": 12.5,             // m/s
        "passenger_id": "c01",
        "destination": "cafe_01",
        "yolo_detections": [      // 최신 카메라 프레임의 YOLO 결과
          { "cls": "pedestrian", "conf": 0.92, "bbox": [120, 80, 50, 110] },
          { "cls": "traffic_light", "conf": 0.88, "bbox": [320, 40, 30, 80], "state": "red" }
        ]
      }
    ],
    "drones": [
      {
        "id": "d01",
        "pos": [180.0, 25.0, 90.0],
        "destination": "cafe_01",
        "cargo": "coffee_beans",
        "battery": 0.78
      }
    ],
    "traffic_lights": [
      { "id": "tl_01", "pos": [200, 0, 200], "state": "green", "remaining_sec": 8 }
    ],
    "traffic_forecast": {
      "next_15min": [
        { "edge_id": "main_st_01", "congestion": 0.42 },
        { "edge_id": "main_st_02", "congestion": 0.78 }
      ]
    }
  }
}

```


event (비정기, 중요 사건 발생 시)

```
{
  "v": 1,
  "type": "event",
  "ts": 1700000000.123,
  "tick": 12345,
  "payload": {
    "kind": "dialog_started" | "dialog_ended" | "reflection_generated"
      | "trip_requested" | "trip_completed" | "collision_avoided"
      | "god_command_executed" | "weather_changed",
    "actors": ["c01", "c03"],
    "location": [120, 0, 80],
    "details": {
      "dialog_id": "d_abc123",
      "topic": "weekend plans"
    }
  }
}
```

dialog_chunk (시민 대화 스트리밍)

```
{
  "v": 1,
  "type": "event",
  "ts": 1700000000.456,
  "tick": 12345,
  "payload": {
    "kind": "dialog_chunk",
    "dialog_id": "d_abc123",
    "speaker_id": "c01",
    "text_delta": "안녕 ",
    "done": false
  }
}
```

memory_added (시민 메모리 패널 실시간 업데이트)

```
{
  "v": 1,
  "type": "event",
  "ts": 1700000000.789,
  "tick": 12345,
  "payload": {
    "kind": "memory_added",
    "citizen_id": "c01",
    "memory": {
      "id": "m_xyz789",
      "type": "observation", // observation | reflection | plan
      "content": "박지훈과 카페에서 만났다",
      "importance": 6,
      "ts": "2026-05-24T08:43:00"
    }
  }
}
```

7.3 클라이언트 → 서버 메시지

god_command (God Mode 음성/텍스트 명령)

```
{
  "v": 1,
  "type": "command",
  "ts": 1700000000.123,
  "payload": {
    "kind": "god_command",
    "input_modality": "voice" | "text",
    "raw_text": "도시에 갑자기 폭우가 내립니다",
    "audio_blob_b64": null,          // 음성 입력시 base64 wav
    "user_id": "presenter"
  }
}
```

select_entity (관찰 대상 변경)

```
{
  "v": 1,
  "type": "command",
  "ts": 1700000000.123,
  "payload": {
    "kind": "select_entity",
    "entity_type": "citizen" | "vehicle" | "drone",
    "entity_id": "c01"
  }
}
```

sim_control (시뮬 제어)

```
{
  "v": 1,
  "type": "command",
  "ts": 1700000000.123,
  "payload": {
    "kind": "sim_control",
    "action": "pause" | "resume" | "set_speed" | "reset",
    "speed_multiplier": 1.0          // 0.5x ~ 100x
  }
}
```

7.4 ack / error

```
// 성공 응답
{ "v": 1, "type": "ack", "ts": ..., "payload": { "command_id": "...", "ok": true } }

// 실패 응답
{
  "v": 1,
  "type": "error",
  "ts": ...,
  "payload": {
    "code": "INVALID_COMMAND" | "INTERNAL" | "RATE_LIMIT",
    "message": "..."
  }
}
```

7.5 압축 및 차분 전송

최적화

전체 상태를 매 tick 60Hz로 보내면 대역폭이 폭발한다. 다음 두 전략으로 압축한다.

- **Delta encoding**: 첫 frame만 full state, 이후엔 변경된 엔티티만 송출 (`state_diff` 메시지)
- **MessagePack**: JSON 대신 binary 직렬화로 30~40% 절감
- **permessage-deflate**: WebSocket 자체 압축 활성화

08. REST API 스펙

WebSocket으로 처리하기 부적합한 무거운 단일 요청은 REST로 분리한다. 모두 `/api/v1` prefix.

메서드	경로	설명
GET	/api/v1/sim/status	현재 시뮬 상태 조회 (tick, 시간, 시민 수 등)
POST	/api/v1/sim/start	시뮬레이션 시작
POST	/api/v1/sim/stop	시뮬레이션 정지
POST	/api/v1/sim/reset	시뮬레이션 초기화 (선택적 seed)
GET	/api/v1/citizens	전체 시민 목록 + 페르소나 요약
GET	/api/v1/citizens/{id}	특정 시민 상세 (페르소나, 관계, 현재 계획)
GET	/api/v1/citizens/{id}/memories	메모리 스트림 페이지네이션 조회
POST	/api/v1/citizens/{id}/reflect	강제 reflection 트리거
GET	/api/v1/timeline	발생한 주요 사건 타임라인
GET	/api/v1/vehicles/{id}/camera	차량 카메라 최신 프레임 (PNG)
POST	/api/v1/god/command	God Mode 명령 (REST 백업 경로)
GET	/api/v1/health	헬스체크 (LLM/Vision/Sim 각각)
GET	/api/v1/metrics	Prometheus 호환 메트릭

8.1 예시 응답: GET /api/v1/citizens/c01

```
{
  "id": "c01",
  "name": "김민지",
  "age": 28,
  "occupation": "바리스타",
  "personality": ["다정함", "수다스러움", "완벽주의"],
  "current_location": "cafe_01",
  "current_action": "커피 내리는 중",
  "relationships": [
    { "target_id": "c03", "type": "friend", "closeness": 0.7 },
    { "target_id": "c07", "type": "neighbor", "closeness": 0.4 }
  ],
  "plan_today": [
    { "time": "07:00", "action": "기상 후 출근 준비" },
    { "time": "08:30", "action": "카페 도착, 오픈 준비" },
    { "time": "12:00", "action": "점심 식사" }
  ],
  "memory_count": 142,
  "last_reflection_at": "2026-05-24T06:00:00"
}
```

09. 데이터 모델

핵심 도메인 모델은 Pydantic으로 정의되어 서버/스키마/문서가 단일 소스에서 생성된다.

9.1 Citizen

```
class Citizen(BaseModel):
    id: str # "c01"
    name: str # "김민지"
    age: int
    gender: Literal["M", "F", "N"]
    occupation: str
    home_location_id: str # 거주지
    workplace_id: str | None
    personality: list[str] # ["다정함", "수다스러움", ...]
    background: str # 자유 텍스트 배경 스토리
    pos: Vec3 # 현재 위치
    rot: float # 현재 회전 (yaw)
    anim_state: AnimState
    current_action: str
    talking_to: str | None
    plan_tree: PlanTree
    memory_store: MemoryStoreRef # ChromaDB collection 참조
    relationships: dict[str, Relationship]
    created_at: datetime
```

9.2 Memory

```
class Memory(BaseModel):
    id: str
    citizen_id: str
    type: Literal["observation", "reflection", "plan"]
    content: str # 자연어 메모리 내용
    importance: int # 1-10, LLM이 평가
    embedding: list[float] | None # bge-m3 벡터 (외부 저장)
    created_at: datetime
    last_accessed_at: datetime
    access_count: int = 0
    related_entity_ids: list[str] # 연관된 시민/장소 ID

class RetrievalQuery(BaseModel):
    citizen_id: str
    query_text: str
    top_k: int = 5
    recency_weight: float = 0.5
    importance_weight: float = 1.0
    relevance_weight: float = 1.0
```

9.3 Plan Tree

```
class PlanNode(BaseModel):
    level: Literal["day", "hour", "block"] # 3단 분해
    start_time: time
    end_time: time
    action: str # "카페에서 일하기"
    location_id: str | None
    sub_plans: list["PlanNode"]
    completed: bool = False

class PlanTree(BaseModel):
    citizen_id: str
    date: date
    root: PlanNode
    replan_history: list[ReplanEvent]
```

9.4 Vehicle

```
class Vehicle(BaseModel):
    id: str
    type: Literal["taxi", "bus", "private"]
    pos: Vec3
    rot: float
    speed: float # m/s
    max_speed: float = 16.0
    acceleration: float = 3.0
    path: list[Vec3] # A* 경로 waypoints
    path_idx: int = 0
    passenger_id: str | None
    destination_location_id: str | None
    camera_fov: float = 90.0
    last_yolo_frame_ts: float
    last_yolo_detections: list[YoloDetection]
```

9.5 YoloDetection

```
class YoloDetection(BaseModel):
    cls: Literal["pedestrian", "vehicle", "traffic_light", "drone", "building"]
    confidence: float
    bbox: tuple[float, float, float, float] # x, y, w, h (normalized)
    state: str | None # 신호등의 경우 "red"|"yellow"|"green"
    distance_estimate: float # ground truth (sim에서 알 수 있음)
```

9.6 Location (도시 시설)

```
class Location(BaseModel):
    id: str # "cafe_01"
    name: str # "코지 카페 강남점"
    type: Literal["home", "cafe", "office", "library", "park", "shop", "restaurant"]
    pos: Vec3
    capacity: int # 동시 수용 인원
    current_visitors: list[str] # 현재 방문 중인 citizen IDs
    operating_hours: tuple[time, time]
    metadata: dict[str, Any] # type별 추가 정보
```

9.7 TrafficLight

```
class TrafficLight(BaseModel):
    id: str
    pos: Vec3
    controlled_edges: list[str] # 이 신호가 제어하는 도로 ID들
    state: Literal["red", "yellow", "green"]
    remaining_sec: float
    cycle_pattern: list[tuple[str, float]] # RL이 학습할 대상
    is_rl_controlled: bool = True
```


10. 모듈 상세 설계

10.1 Citizen Agent NLP

책임: 각 시민의 의사결정, 메모리 관리, 대화, 계획 수립.

핵심 알고리즘: Memory Retrieval Score

Generative Agents 논문의 retrieval 공식을 그대로 사용:

```
score(m, q) =  $\alpha \cdot \text{recency}(m)$ 
              +  $\beta \cdot \text{importance}(m)$ 
              +  $\gamma \cdot \text{relevance}(m, q)$ 

where:
  recency(m)   =  $\exp(-\text{decay} \cdot \text{hours\_since\_last\_access})$ 
  importance(m) =  $m.\text{importance} / 10$ 
  relevance(m, q) =  $\text{cosine\_similarity}(\text{embed}(m.\text{content}), \text{embed}(q))$ 

 $\alpha = 0.5, \beta = 1.0, \gamma = 1.0$  (튜닝 가능)
```

Reflection Trigger

- 누적 importance sum이 임계값 (default: 150)을 넘으면 reflection 수행
- 최근 100개 메모리를 LLM에 전달, "이 경험들로부터 도출할 수 있는 상위 통찰 5개" 추출
- 생성된 통찰은 `type: reflection` 메모리로 저장

Plan Tree 생성

- **Day 계획:** 매일 00:00에 LLM 호출, 24시간 큰 그림 (12 블록)
- **Hour 계획:** 현재 day 블록을 1시간 단위 5개 sub-action으로 분해
- **Block 계획:** 현재 hour를 10분 단위로 분해 (필요시만)
- 예외 발생 시(다른 시민과 만남, 날씨 변화 등) replan

10.2 Vehicle Control ROBOT

책임: 차량의 경로 계획, 추종 제어, 충돌 회피, 시민 픽업/드롭.

Kinematic Vehicle Model

```
# 매 tick 업데이트 (dt = 0.1s)
target_speed = lookup_speed_limit(current_edge)
if needs_to_brake:          # YOLO에서 pedestrian/red light 감지
    target_speed = 0.0
elif distance_to_next_waypoint < 5.0:
    target_speed *= 0.5

# PID-like simple acceleration
delta = target_speed - current_speed
acceleration = clamp(delta * 2.0, -max_decel, max_accel)
current_speed += acceleration * dt
pos += direction * current_speed * dt
```

경로 계획

- 도시 도로망을 그래프로 모델링 (NetworkX), 노드 = 교차로, 엣지 = 도로
- A* 알고리즘으로 출발지 → 목적지 최단 경로
- 매 30초마다 트래픽 예측 반영해 경로 재계산 (선택)

YOLO 통합

- 차량 전방 카메라를 Three.js에서 별도 render target으로 캡처 (1초당 5프레임)
- WebSocket으로 base64 JPEG 전송 → Vision Service → YOLO 결과 반환
- 결과를 Vehicle 상태에 저장 → 다음 tick 의사결정에 반영

10.3 Vision Service VISION

책임: YOLOv8 기반 실시간 객체 탐지, 학습 데이터 자동 생성.

학습 데이터 자동 생성 파이프라인

1. Three.js scene에서 다양한 카메라 위치/각도로 5000장 렌더링
2. 각 프레임에 대해 sim engine의 ground truth 객체 위치를 2D 화면 좌표로 투영 (camera projection matrix)
3. YOLO 포맷 (.txt) 라벨 파일 자동 생성
4. train/val 80/20 분할
5. 동일 환경에서 YOLOv8n fine-tune (20 epochs)

추론 서비스

- FastAPI + ONNX Runtime GPU
- 입력: base64 인코딩 JPEG, 출력: detections 배열
- 배치 처리로 throughput 향상 (4~8 frames per batch)
- 목표 throughput: 50 FPS (차량 5대 × 5fps × safety margin)

평가 지표

- $mAP@0.5 \geq 0.85$ 목표 (자동 라벨 정답 vs YOLO 예측)
- End-to-end latency $\leq 80ms$ (전송 + 추론 + 응답)

10.4 Traffic AI ML/DL

책임: 트래픽 패턴 시계열 예측, 신호등 제어 강화학습.

시계열 예측 (LSTM)

- **입력:** 각 도로 엣지의 최근 60분 trip count (10분 단위 6 timesteps × 도로 수)
- **출력:** 향후 15분 congestion level 예측 (0~1)
- **모델:** 2-layer LSTM, hidden 64, dropout 0.2
- **학습 데이터:** 시물을 1주일(=가속하면 30분) 돌려서 자동 수집

신호등 RL (PPO)

```
gym.Env: TrafficSignalEnv
observation_space:
  - 각 신호등 인접 도로의 큐 길이 [0, 50]
  - 현재 신호 상태 (one-hot)
  - 시간대 (sin/cos encoding)
action_space:
  - Discrete: 각 신호등별 (KEEP / SWITCH_NEXT)
reward:
  - -sum(queue_length) - 0.1 * num_switches
```

- Stable-Baselines3 PPO, 1M timesteps 사전학습
- 학습된 정책은 추론 전용으로 로드 → 시물 중에는 forward만
- 비교 baseline: 고정 주기 신호 (60s 주기)

10.5 God Mode NLP

책임: 발표자의 음성/텍스트 명령을 받아 월드에 개입.

처리 파이프라인

```
[Browser Mic]
  → Web Audio API (16kHz, mono)
  → MediaRecorder (1초 침크)
  → WebSocket binary 전송
  → faster-whisper (Cloud) STT
  → LLM intent parser ("어떤 종류의 개입인가?")
  → Command Dispatcher
    → weather_change: World.weather 직접 수정
    → spawn_event: 모든 시민의 메모리에 observation 주입
    → modify_citizen: 특정 시민의 plan/mood 변경
    → toggle_infrastructure: 신호등/도로 비활성화
```

지원 명령 카테고리

카테고리	예시
환경	"폭우가 내려" / "해가 진다" / "안개가 짙어진다"
이벤트	"광장에서 콘서트가 시작됐다" / "정전이 발생했다"
인물 변경	"김민지가 갑자기 복권에 당첨됐다" / "박지훈이 우울하다"
인프라	"모든 신호등이 고장났다" / "메인 도로가 폐쇄됐다"
관계	"이수현과 박지훈이 어제 다툰 적이 있다" (메모리 주입)

11. 데이터 · 라벨링 전략

본 프로젝트의 핵심 차별점은 **모든 학습 데이터를 자체 생성**한다는 점이다. 사람이 라벨링하는 작업은 0이며, 이는 LLM과 시뮬레이션 환경의 결합을 통해 달성된다.

데이터 종류	생성 주체	방법	수량
시민 페르소나	LLM (1회)	seed prompt로 20개 페르소나 일괄 생성 → 일관성 검증 → JSON 저장	20
도시 시설명/스토리	LLM (1회)	도시 맵 절차 생성 후 각 건물에 이름/용도 부여	~50
YOLO 학습 이미지	Three.js 렌더 (자동)	다양한 시간/날씨/카메라 각도로 자동 캡처	5,000
YOLO 라벨	Sim Engine (자동)	ground truth 3D 위치 → 카메라 행렬로 2D bbox 자동 투영	5,000
트래픽 시계열	시뮬 자체	1주일 시뮬 가속 실행하며 도로별 trip count 수집	~10K row
RL 학습 데이터	RL 학습 루프	TrafficSignalEnv에서 self-play로 1M timesteps 자동 생성	1M
시민 대화 코퍼스	시뮬 자체	시뮬 진행 자체가 자연어 대화 데이터셋이 됨	자동 누적
평가용 reflection 품질 점수	LLM-as-judge	생성된 reflection에 대해 GPT-4o가 일관성/통찰력 1-10 채점	샘플 100

핵심 트릭: Sim-to-Label Loop

시뮬레이션은 모든 객체의 ground truth 상태(위치, 종류, 관계)를 알고 있다. 이를 활용하면 YOLO 학습 라벨이 공짜로 무한 생성된다. labellmg 같은 도구로 사람이 박스를 그리는 작업이 완전히 사라진다.

12. 개발 마일스톤

기간 제약 없이 품질 우선으로 설계한 7단계 마일스톤. 각 마일스톤은 독립적으로 데모 가능한 상태(playable state)를 산출한다.

M0 · Foundation

"hello world"가 클라우드 → 브라우저로 흐른다

목표

- 모노레포 셋업 (pnpm + uv workspace)
- RunPod에 vLLM 컨테이너 배포, OpenAI API 호환 엔드포인트 검증
- FastAPI Orchestrator + Socket.IO 허브 골격
- Next.js + R3F 클라이언트 부트스트랩 (빈 씰에 큐브 하나)
- WebSocket으로 서버 → 클라이언트 ping/pong 60Hz 송신
- shared-schemas 패키지 + Pydantic → Zod 자동 생성 파이프라인

Definition of Done

- 로컬 브라우저에서 클라우드 vLLM에 "안녕" 요청 → 한국어 응답 표시
- tick counter가 클라우드에서 송신되어 브라우저에 1초마다 표시

M1 · City Scene

예쁜 3D 빈 도시가 브라우저에 렌더된다

목표

- 도시 절차적 생성 스크립트 (격자형 도로 + 건물 배치)
- Kenney City Kit GLB 에셋 통합
- Day/Night cycle 셰이더, Sky from drei
- OrbitControls + 시민/차량 클릭 시 카메라 follow
- PostFX (Bloom, SSAO) 적용
- LLM이 50개 시설에 이름/용도 자동 부여

Definition of Done

- 발표자가 카메라를 자유롭게 움직이며 도시를 둘러볼 수 있음
- 시간대를 슬라이더로 조절 시 라이팅이 부드럽게 변화

목표

- LLM으로 20개 페르소나 자동 생성, JSON 저장
- Memory Stream + ChromaDB 통합 + retrieval 알고리즘
- Plan tree 3단 분해 알고리즘
- 시민 간 만남 → 대화 트리거, 양쪽 메모리 동시 업데이트
- Reflection 알고리즘 (importance 누적 임계 기반)
- Three.js에 캐릭터 모델 + Mixamo 애니메이션 통합
- Memory Panel UI (사이드바)

Definition of Done

- 시물 가상 1일을 돌리면 시민들이 각자 일과를 수행
- 두 시민이 마주치면 자동 대화 → 메모리 누적 확인
- 특정 시민 클릭 시 우측에 memory stream이 실시간 표시

목표

- 차량 5대 + 드론 2대 추가
- NetworkX 도로 그래프 + A* 경로 계획
- Kinematic vehicle model + 충돌 회피
- Trip manager: 시민 호출 → 차량 매칭 → 픽업/드롭
- Vision Service: YOLOv8 ONNX 추론 + FastAPI
- YOLO 학습 데이터 자동 생성 스크립트 + 학습 완료
- Vehicle Cam Panel UI (YOLO 박스 오버레이)

Definition of Done

- 시민이 카페로 가야 할 때 자동으로 택시 호출 → 픽업 → 드롭
- 도로에 보행자 출현 시 차량이 감속/정지
- 차량 카메라 패널에 실시간 YOLO 박스 표시
- YOLO mAP@0.5 $\geq$ 0.85 달성

목표

- TrafficSignalEnv (gym.Env 호환) 구현
- PPO 1M timesteps 사전학습 (4090 GPU 활용)
- 학습된 정책 추론 wrapper → 시뮬에 통합
- LSTM 트래픽 예측 학습 + 추론 통합
- Traffic Chart Panel UI (예측값 vs 실측값)
- Baseline 비교 (고정 60초 주기 vs RL 정책)

Definition of Done

- 출근 러시 시뮬에서 RL 신호등이 baseline 대비 평균 대기시간 20%+ 감소
- LSTM 예측 MAPE $\leq 15\%$
- 차트 패널에서 예측-실측 동기화 시각화

목표

- Web Audio + MediaRecorder 마이크 캡처
- faster-whisper STT 통합 (Cloud)
- LLM intent parser → Command Dispatcher
- 5개 명령 카테고리 핸들러 구현 (날씨/이벤트/인물/인프라/관계)
- God Mode 명령 → 시민 메모리 주입 → 후속 행동 변화 검증
- God Mode 마이크 UI 컴포넌트

Definition of Done

- "폭우가 내린다" 외침 → 5초 안에 화면 비 효과 + 차량 감속
- "김민지가 복권에 당첨됐다" → 김민지가 다음 만나는 시민에게 자랑함
- 음성 → 명령 실행까지 end-to-end latency ≤ 3 초

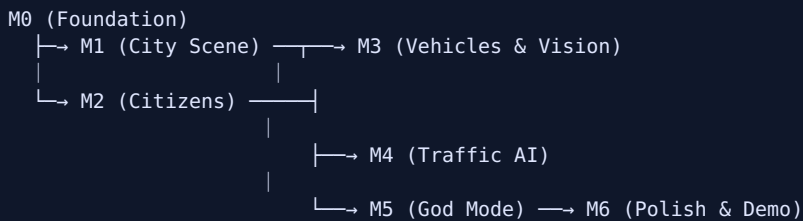
목표

- 전체 시스템 부하 테스트 (4시간 연속 가동)
- 리플레이 모드 (오프라인 폴백) 구현
- 녹화된 데모 영상 3편 (각 1~2분)
- 발표용 슬라이드 + 시연 스크립트
- GitHub README + 데모 GIF + 아키텍처 SVG
- 발표 리허설 5회 이상
- 네트워크 끊김 대비 (테더링 백업, snapshot 복구)

Definition of Done

- 인터넷 끊겨도 5초 안에 리플레이 모드로 자동 전환
- 15분 풀 라이브 시연 무사고 완주 (3회 연속)

12.8 마일스톤 간 의존성



M1과 M2는 병렬 진행 가능. M3는 M1+M2 모두 필요. M4와 M5는 M3 이후 병렬 가능. M6는 마지막에 통합.

13. 인프라 · 배포

13.1 클라우드 GPU 선택

제공자	GPU	시간당 비용	장단점
RunPod	RTX 4090 24GB	\$0.34 (spot) / \$0.44 (on-demand)	한국 region 가까움(JP), 안정적, Docker template 지원. 1순위 추천
Vast.ai	RTX 4090 24GB	\$0.30~0.50	최저가, 다만 instance 안정성 들쭉날쭉
Lambda Labs	A10G 24GB	\$0.75	매우 안정, 발표 당일 백업 추천
Hyperstack	RTX 4090	\$0.50	유럽 region 중심

운용 전략

개발 단계는 RunPod spot으로 비용 최소화 ($\$0.34/h \times \text{일 8시간} \times 30\text{일} = \82). 발표 1주일 전부터는 on-demand로 전환해 안정성 확보. 발표 당일은 Lambda Labs A10G 백업 인스턴스를 워밍업.

13.2 Docker Compose 구성

```
# docker-compose.yml (cloud server)
services:
  vllm:
    image: vllm/vllm-openai:latest
    runtime: nvidia
    environment:
      - MODEL=Qwen/Qwen2.5-14B-Instruct-AWQ
      - QUANTIZATION=awq
    ports: ["8000:8000"]
    volumes: ["/models:/root/.cache/huggingface"]
    deploy:
      resources:
        reservations:
          devices: [{ driver: nvidia, count: 1, capabilities: [gpu] }]

  vision:
    build: { context: ./server, dockerfile: ../infra/docker/vision.Dockerfile }
    ports: ["8001:8001"]
    depends_on: [vllm]
    runtime: nvidia
    environment:
      - YOLO_MODEL=/app/models/yolov8n_city.onnx

  orchestrator:
    build: { context: ./server, dockerfile: ../infra/docker/orchestrator.Dockerfile }
    ports: ["8080:8080"]
    depends_on: [vllm, vision, redis]
    environment:
      - VLLM_URL=http://vllm:8000/v1
      - VISION_URL=http://vision:8001
      - REDIS_URL=redis://redis:6379

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  caddy:
    image: caddy:2
    ports: ["443:443"]
    volumes: ["/infra/caddy/Caddyfile:/etc/caddy/Caddyfile"]
    # TLS 중단, WSS 변환
```

13.3 통신 보안

- **TLS:** Caddy로 자동 Let's Encrypt 인증서 발급, WSS 강제
- **인증:** JWT bearer token (단일 발표자 user)
- **Rate Limiting:** God Mode 명령은 분당 30회 제한
- **CORS:** localhost + Vercel deploy URL만 허용

13.4 클라이언트 배포

- **개발:** `pnpm dev` on localhost:3000
- **배포:** Vercel (자동 CI/CD from GitHub main)

- **발표용**: 노트북 로컬 build + 오프라인 캐시 (PWA)

13.5 모니터링 & 로깅

- 구조화된 로그 (loguru JSON) → CloudWatch / Loki
- Prometheus metrics (LLM latency, vision throughput, tick rate)
- Sentry로 클라이언트 에러 캡처

14. 리스크 매트릭스

리스크	발생 확률	영향도	대응책
발표 중 인터넷 끊김	중	치명적	(1) 사전 녹화 영상 3편 백업 (2) 로컬 리플레이 모드 자동 전환 (3) 폰 테더링 준비
vLLM 추론 latency 과대	중	높음	(1) Plan caching으로 호출 빈도 1/100로 감소 (2) 14B → 7B 폴백 옵션 (3) 시간 가속으로 절대 호출 수 감소
클라우드 인스턴스 중단 (spot)	높음	중	(1) 발표 1주 전부터 on-demand 전환 (2) snapshot 복구 자동화
LLM이 부적절한 발화 생성	중	중	(1) 시스템 프롬프트로 안전 가이드라인 (2) 출력 필터 (3) 시연 시나리오 사전 시드 고정
20개 시민 동시성 데드락	중	높음	(1) asyncio.Semaphore로 동시 호출 제한 (2) lock-free 메시지 큐 설계 (3) 부하 테스트로 사전 검증
YOLO 학습 데이터 품질 낮음	낮	중	(1) 데이터 augmentation (회전, 밝기, 노이즈) (2) 검증셋 수동 샘플링 확인
RL 신호등 baseline에 못 미침	중	낮	(1) Reward shaping 추가 (2) 학습 timesteps 5M까지 증가 (3) 데모에서는 학습 곡선만 보여줘도 충분
Three.js 60fps 미달 (저 사양 노트북)	중	중	(1) LOD (Level of Detail) 적용 (2) 시민 수 동적 조절 (3) 그림자 품질 자동 다운그레이드
음성 인식 실패 (발표장 소음)	중	낮	(1) 헤드셋 마이크 사용 (2) 텍스트 입력 백업 UI (3) 사전 설정한 매크로 버튼
비용 초과	낮	낮	(1) 일일 비용 알림 (\$10/day cap) (2) 미사용 시간 자동 인스턴스 중지 cron

15. 평가 지표

15.1 시스템 지표

TICK RATE ≥ 10 Hz	WS BROADCAST LATENCY ≤ 50 ms	YOLO THROUGHPUT ≥ 50 FPS
LLM TPS (PER REQUEST) ≥ 40 tok/s	CLIENT FPS ≥ 60 FPS	GOD CMD → EFFECT ≤ 3 sec

15.2 AI 성능 지표

지표	목표	측정 방법
YOLO mAP@0.5	≥ 0.85	자동 생성 validation set 1000장
LSTM 트래픽 예측 MAPE	≤ 15%	15분 후 congestion 예측 vs 실측
RL 신호등 평균 대기 감소	≥ 20%	vs 60초 고정 주기 baseline
Reflection 통찰력 점수	≥ 7.5 / 10	GPT-4o LLM-judge 평가
시민 일관성 점수	≥ 8.0 / 10	페르소나 vs 행동 일치도 LLM 평가

15.3 시연 품질 지표

- 15분 시연 무사고 완주율 100%
- God Mode 명령 성공률 ≥ 90% (10회 중 9회 의도대로 실행)
- 관객 참여 인터랙션 ≥ 3회

16. 발표 시연 시나리오

16.1 시간 배분 (총 15분)

시간	단계	내용
0:00-2:00	오프닝	프로젝트 소개, 4분할 화면 레이아웃 안내, "이건 GTA가 아닙니다"
2:00-6:00	자율 일과	시물 시간 06:00→09:00 가속, 시민 자동 출근, 차량 YOLO, 메모리 스트림 실시간 표출
6:00-9:00	God Mode	"폭우가 내린다", "김민지가 복권 당첨" 등 발표자 음성 개입 시연
9:00-12:00	관객 참여	관객이 외친 명령을 발표자가 마이크에 전달 → 시물 반응 관찰
12:00-14:00	1주일 영상	사전 녹화된 가속 영상 + emergent behavior 사례 5개 소개
14:00-15:00	마무리	아키텍처 다이어그램 + 학술 레퍼런스 + Q&A 안내

16.2 화면 레이아웃



16.3 핵심 발화 (대본)

오프닝

"여러분이 지금 보시는 이 도시는 제가 미리 짜놓은 시나리오가 아닙니다. 25명의 AI 시민, 5대의 자율주행 차량, 학습된 신호등이 지금 이 순간 자기들끼리 결정해서 움직이고 있습니다. 그리고 모든 AI 추론은 제 노트북이 아니라 서울 데이터센터의 GPU에서 실시간으로 처리되고 있습니다."

God Mode 직전

"지금부터 제가 신이 되어보겠습니다. 마이크에 명령하면 도시에 그 일이 실제로 일어나고, 시민들의 기억 속에 저장됩니다. 그리고 그 기억이 내일의 행동을 바꿉니다."

마무리

"본 프로젝트는 Stanford의 Generative Agents 논문, YOLOv8 객체인식, PPO 강화학습 신호등, LSTM 시계열 예측을 단일 시스템으로 통합했습니다. 학습 데이터는 0개도 사람이 라벨링하지 않았습니다. 코드는 GitHub에 공개되어 있고, QR 코드는 여기 있습니다."

17. 향후 확장

확장 방향	설명
경제 시스템	각 시민에게 가상 화폐, 가게 매출, 가격 결정 등을 추가하여 경제 시뮬레이션
학습형 차량	차량도 LLM 에이전트화하여 "오늘 어디 가고 싶다"를 자율 결정
다국어 시민	시민마다 모국어를 다르게 부여하여 다언어 대화 시뮬
VR 모드	WebXR로 발표자가 도시 안으로 들어가서 1인칭으로 시민과 대화
도시 간 연결	여러 도시 인스턴스를 연방화하여 시민 이주, 교역 시뮬
실시간 협업	여러 관전자도 동시 접속, 각자 다른 시민을 관찰/조작
모바일 클라이언트	React Native로 폰에서 모니터링 가능한 동반 앱
블록체인 통합	시민의 주요 사건/계약을 온체인 기록 (NFT 사회 시뮬)

18. 참고 문헌

1. Park, J.S., O'Brien, J.C., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST '23.
2. Jocher, G., Chaurasia, A., Qiu, J. (2023). YOLO by Ultralytics (Version 8.0.0). GitHub.
3. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
4. Kwon, W., Li, Z., Zhuang, S., et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP '23 (vLLM).
5. Hochreiter, S., Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.
6. Hart, P.E., Nilsson, N.J., Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE.
7. Qwen Team (2024). Qwen2.5 Technical Report. Alibaba Cloud.
8. Cattaneo, F., et al. (2022). Darts: User-Friendly Modern Machine Learning for Time Series. JMLR.
9. OpenAI (2022). Whisper: Robust Speech Recognition via Large-Scale Weak Supervision.
10. Raffin, A., et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. JMLR.

— End of Document —

Project Aetherville · PRD v1.0 · 2026